# Chapter Five
# Test Metrics

# What is a Metric?

- Metrics can be defined as “STANDARDS OF MEASUREMENT.”
  - A metric is a unit used for describing or measuring an attribute
- Test metrics are the means by which the software quality can be measured
- Test metrics provide visibility into the readiness of the product, and give a clear measurement of the quality and completeness of the product

# Testing Metrics

- "We cannot improve what we cannot measure," and Test Metrics helps us to do the same.
- **The goal of software testing metrics is:**
  - to improve the efficiency and effectiveness of the software testing process, and
  - help make better decisions for the further testing process by providing reliable data about the testing process.

# Why Test Metrics

- The aim of collecting **test metrics** is to use the data for **improving the test process**. This includes finding answers to the questions like:

| | |
|---|---|
| – How long will it take to test?<br>– How much money will it take to test?<br>– How bad are the bugs?<br>– How many bugs found were fixed? reopened? closed? deferred?<br>– How many bugs did the test team not find?<br>– How much of the software was tested? | – Will testing be done on time? Can the software be shipped on time?<br>– How good were the tests? Are we using low-value test cases?<br>– Was the test effort adequate? Could we have fit more testing in this release? |

# Pre-process Metrics: Estimation

- These metrics are used to estimate and plan the testing process before it begins.
- They help set expectations and provide a baseline for monitoring progress.

# Examples of Pre-process Metrics

- **Estimated Test Case Preparation Effort**: Time anticipated to write and review test cases.
- **Estimated Testing Effort**: Total time forecasted for executing all test cases.
- **Projected Defect Density**: Expected number of defects per module or functionality.
- **Risk Assessment Metrics**: Identifying and quantifying potential risks in the software.
- **Resource Allocation Metrics**: Number of testers, test environments, and tools required.

# Examples

- **Estimated Test Case Preparation Effort**
  - Estimated Effort = (Number of Test Cases) × (Average time per test case) + Review & rework buffer
- **Example Scenario**
  - You are testing a **"Password Reset" feature**. Based on requirements, you have identified **40 test cases** (positive, negative, boundary, security, usability, etc.). From historical data or expert judgment: Writing a single test case (steps, inputs, expected results) takes **15 minutes** on average (0.25 hours).
  - *Note: This includes initial drafting but not yet peer review.*
  - Your team does a peer review for all test cases. Typically, reviewing and fixing comments adds **20%** extra time. What would be the estimated test case preparation effort? Given that a person works 8hr/day, how many people per day are needed?

# Estimated testing effort

- 1. Basic Test Execution Calculation

$$\text{Total Effort} = \text{Total Test Cases} \times \text{Average Time Per Test Case}$$

  - Average Time per Test Case: Includes setup, execution, and logging results (e.g., 10-15 minutes per manual test case is a common starting point).
- 2. Three-Point Estimation (Recommended for accuracy)
- This formula accounts for uncertainty and complexity by considering best/worst-case scenarios:

$$\text{Effort (E)} = \frac{O + 4M + P}{6}$$

- O (Optimistic): Best case (ideal conditions, no bugs).
- M (Most Likely): Most likely scenario.
- P (Pessimistic): Worst case (unstable environment, many bugs found).

- Example:
  - Analyze and Organize Test Cases: Group test cases by type (Functional, Regression, Security) and complexity (Low, Medium, High).
  - Estimate Time Per Type: Assign an estimated time to each complexity level.
    - Low: 5 mins/test case
    - Medium: 15 mins/test case
    - High: 30+ mins/test case
  - Calculate Sub-Totals: (Number of Low Cases x*5) + (Number of Medium Cases * 15) ...
  - Add Contingency/Buffer: Always add a 20-25% buffer for activities like environment setup, bug verification, and meetings.

$$\text{Total Time} = (\text{Execution Time} + \text{Preparation Time}) \times 1.25$$

  - Convert to Person-Days:

# Simple stage-based projection

- **Projected defect density**:
  - Defect Density = Estimated Number of defects / Size of the software (e.g., per KLOC or per function point).
    - Projected Defect Density is an estimate of what the defect density will be at a future point (e.g., after testing is complete) based on early data, historical trends, or statistical models.
  - It helps you:
    - Decide if a module is ready for release.
    - Compare quality across teams or projects.
    - Predict remaining defects and required rework effort

- **Example**
  - You are testing a module of 10,000 lines of code (10 KLOC).
  - After the first round of testing, you find 30 defects.
  - From past projects, you know that your testing process typically finds 75% of all defects during this first round.

- **Step 1 – Estimate total defects:**
  - Estimated total defects = Defects found so far / Detection rate at this stage
    = 30 / 0.75 = 40 defects
- **Step 2 – Compute projected defect density:**
  - Projected Defect Density = Estimated total defects / Size (KLOC)
    = 40 defects / 10 KLOC = 4 defects per KLOC
- **Interpretation:**
  - When testing finishes, you expect about 4 defects per thousand lines of code in this module

# Capture-recapture

- You have two independent test efforts on the same version
- **Step 1:**
  - Estimiated Total = (defects_found_by_A * defects_found_by_B) / defects_found_by_both
- **Step 2:**
  - Then we do the defect density prediction

- Example:
  - Two testers work independently on the same 5 KLOC module:
  - Tester A finds 20 defects
  - Tester B finds 15 defects
  - Defects found by **both** = 5

Projected total defects = (20 × 15) / 5 = 60

Projected Defect Density = 60 / 5 = 12 defects per KLOC

# Risk Assessment Metrics

- **Risk Exposure**
  - Risk Exposure = Probability of Failure × Impact
    - Probability- Likelihood that a software component contains a critical defect
    - Impact- Business/technical consequence if that defect escapes to production. $, hours, or 1–10
- **Risk Priority Number (RPN)**
  - RPN = Severity × Occurrence × Detection
  - All factors rated 1–10 before test design.
    - Severity- Impact if defect reaches production
    - Occurrence- Likelihood defect exists
    - Detection- How hard it is to catch with planned tests
  - Using the RPN we can take  a decision like
    - If RPN > 150, require automated + exploratory + security tests. If RPN < 30, smoke test only.

# Resource Allocation Metrics

- The testing estimation formula can be used directly to calculate the total effort, which can then be distributed among available resources in the Resource Allocation Matrix.
- Testing Estimation→Total Effort→Resource Allocation Matrix

# In-process Metrics: Process Management

- These metrics monitor the testing process in real-time, ensuring it is on track and identifying areas requiring adjustments.
- They are used for ongoing process control and management.

# Examples of In-process Metrics

- **Test Execution Progress**: Percentage of executed vs. planned test cases.
- **Defect Discovery Rate**: Number of defects identified per day or per test cycle.
- **Defect Severity Distribution**: Classification of defects by criticality (e.g., high, medium, low).
- **Defect Rejection Rate**: Percentage of reported defects that are deemed invalid.
- **Blocked Test Cases**: Number of test cases that could not be executed due to dependencies.
- **Test Environment Availability**: Percentage of time the test environment is up and running.

# Test case execution progress

- Test Execution Progress (%) = (Number of Executed Test Cases / Total Planned Test Cases) × 100
- Example:
  - You are testing a "Login" feature. You have planned 120 test cases for this cycle. By midday yesterday, your team had executed **78** of them.
  - Progress = (78 / 120) × 100 = 65%
  - 65% of the planned test execution is complete. You can estimate the remaining effort or predict whether you will finish on time.

# Defect Discovery Rate

- Defect Discovery Rate = Number of defects identified / Time period
- Where the time period can be:
  - Per day = defects per day (daily rate)
  - Per test cycle = total defects in one complete test cycle
  - Per hour / per person-hour (for team productivity)

**Example – Per Day**

During a 5-day test execution cycle, your team finds defects as follows:

| Day | Defects found |
|---|---|
| 1 | 12 |
| 2 | 18 |
| 3 | 15 |
| 4 | 8 |
| 5 | 5 |

Average daily rate = (12+18+15+8+5) / 5 = 58 / 5 = 11.6 defects/day

The rate starts high, then declines – indicates testing is maturing and remaining defects are harder to find.

- **Example :Per Test Cycle**
  - A full regression test cycle (3 weeks) identifies 87 defects in total.
  - **Calculation:**
    - Defect Discovery Rate = 87 defects per cycle
  - **Interpretation:**
    - Compare across cycles. If next cycle finds only 45 defects, quality has improved (or test coverage decreased).

- **Example: Per Person-Day (Team Productivity)**
  - 4 testers work 5 days (total 20 person-days). They find 60 defects.
  - **Calculation:**
    - Rate = 60 defects / 20 person-days = 3 defects per person-day
  - **Interpretation:**
    - Use to benchmark team efficiency or estimate how many testers needed for future projects.

# Defect Severity Distribution

- Percentage breakdown of defects by their severity levels within a specific software build, release, or testing phase.

$$\text{Severity Distribution} = \left( \frac{\text{Number of defects at severity level } S}{\text{Total number of defects}} \right) \times 100\%$$

- Where S represents a specific severity category (e.g., Critical, High, Medium, Low).

# Example

- Assume your current test cycle found 50 total defects with this breakdown:
  - Severity Count
  - Critical 5
  - Major 15
  - Minor 20
  - Trivial 10
- Critical: (5 ÷ 50) × 100 = **10%**
- Major: (15 ÷ 50) × 100 = **30%**
- Minor: (20 ÷ 50) × 100 = **40%**
- Trivial: (10 ÷ 50) × 100 = **20%**

- Some interpretation of the result can be

| Pattern | What It Indicates | Action |
|---|---|---|
| High % of Critical/Major (>40-50%) | Fundamental design/code issues | Stop new features; focus on root cause analysis & fixing |
| Critical % increasing over time | Regression or unstable codebase | Investigate recent changes; strengthen code reviews |
| Trivial % > 30% | Possibly over-reporting minor UI/text issues | Re-evaluate severity guidelines; train testers |
| Balanced distribution (e.g., 10/30/40/20) | Normal for mid-stage testing | Continue as planned |

- For accuracy
  - **Define severity levels consistently before measuring** – what "Critical" means must be agreed (e.g., system crash/data loss vs. cosmetic issue).
  - **Measure repeatedly** (e.g., weekly) to see trends, not just a single snapshot.
  - **Normalize by test effort** – more testing naturally finds more low-severity defects, skewing distribution.
  - **Combine with defect density** – a product with 1% Critical defects but 100 total defects may be healthier than one with 5% Critical and only 20 total defects.

# Defect Rejection Rate

- Percentage of defects that are rejected (returned) by the development team because they are invalid, duplicate, not reproducible, or not actually defects. It reflects the quality of defect reporting and the alignment between testers and developers.

$$\text{Defect Rejection Rate} = \left( \frac{\text{Number of rejected defects}}{\text{Total number of defects submitted}} \right) \times 100\%$$

- Suppose during a two-week sprint, testers submit 120 defects in total. Developers review them and reject 18 (e.g., 10 duplicates, 5 not reproducible, 3 user environment issues).
- Rejection Rate=( 18/120 )×100=15%

- **For accuracy**
  - Define “rejected” clearly – Distinguish between:
    - Rejected (invalid) – Not a real defect (e.g., tester error, environment misconfiguration).
    - Duplicate – Already reported.
    - Cannot reproduce – Developer tried but could not replicate; may need more info.
      - Some teams count duplicates separately for deeper analysis.
  - Do not penalize testers blindly – A high rejection rate may reflect poor requirements, not lazy testing. Always analyze the reasons for rejection.
  - Combine with defect age / reopen rate – A low rejection rate but high reopen rate means defects are accepted prematurely without proper validation.

# Blocked Test Cases

- Percentage of test cases that cannot be executed because a dependency, environment issue, missing data, or unresolved defect prevents them from running. It helps assess testing efficiency and identify bottlenecks during test execution.

$$\text{Blocked Test Cases Rate} = \left( \frac{\text{Number of blocked test cases}}{\text{Total number of test cases planned for execution}} \right) \times 100\%$$

# Test Environment Availability

- Percentage of time the test environment is operational and accessible for test execution, relative to the total time testers need it.

$$\text{Test Environment Availability} = \left( \frac{\text{Total available time} - \text{Downtime (unplanned)}}{\text{Total available time}} \right) \times \ 100\%$$

- **Total available time** = the total clock time during which testers intend to use the environment
- **Downtime (unplanned) =** any period when the environment is not usable due to crashes, network failures, data corruption, configuration errors, or infrastructure issues.

# Activity

- You are the Test Lead for a mobile banking app undergoing system testing. The test cycle is planned for 5 days (Monday to Friday, 8 hours per day). Below is the data collected during the week. Calculate all six in-process metrics and interpret the results.

| Day | Planned Test Cases | Executed Test Cases | Blocked Test Cases (newly identified that day) | Cumulative Blocked (still unresolved) |
|---|---|---|---|---|
| Mon | 50 | 40 | 5 | 5 |
| Tue | 50 (cumulative 100) | 35 | 10 | 12 (5 from Mon + 7 new, 3 resolved) |
| Wed | 50 (cumulative 150) | 30 | 8 | 15 (12 + 8 – 5 resolved) |
| Thu | 50 (cumulative 200) | 25 | 6 | 18 (15 + 6 – 3 resolved) |
| Fri | 50 (cumulative 250) | 20 | 4 | 20 (18 + 4 – 2 resolved) |

# Defect Data

| Day | Defects Logged | Rejected (Invalid/Duplicate) | Severity Breakdown (of logged defects) |
|---|---|---|---|
| Mon | 12 | 2 | High: 4, Medium: 5, Low: 3 |
| Tue | 15 | 3 | High: 6, Medium: 6, Low: 3 |
| Wed | 10 | 1 | High: 3, Medium: 4, Low: 3 |
| Thu | 8 | 2 | High: 2, Medium: 3, Low: 3 |
| Fri | 6 | 1 | High: 1, Medium: 2, Low: 3 |

- Test Environment Availability
- The test environment is available from 9 AM to 5 PM (8 hours) each day. Unplanned downtime recorded:

| Day | Downtime (minutes) | Cause |
|---|---|---|
| Mon | 30 | Database connection loss |
| Tue | 0 | – |
| Wed | 90 | Network outage + server reboot |
| Thu | 20 | License server expired |
| Fri | 45 | Disk full, cleanup required |

- Calculate:
  - **Test Execution Progress** – Calculate the cumulative percentage of executed vs. planned test cases at the end of the week. Also, calculate daily progress.
  - **Defect Discovery Rate** – Compute the number of defects identified per day and the average for the week (using valid defects only, i.e., excluding rejected).
  - **Defect Severity Distribution** – For the entire week, calculate the percentage distribution of High, Medium, Low severity defects.

- **Defect Rejection Rate** – Calculate the daily and weekly rejection rate.
- **Blocked Test Cases** – Calculate the percentage of planned test cases that became blocked (cumulative) at the end of the week. Also calculate the average daily blocked rate.
- **Test Environment Availability** – Calculate the daily and weekly availability percentage (excluding planned downtime; none given, so total available = 8 hours/day = 480 minutes).

# End-process Metrics: Process Improvement

- These metrics evaluate the overall effectiveness of the testing process after its completion.
- They provide insights to refine and improve future testing cycles.

# Examples of End-process Metrics

- **Defect Removal Efficiency (DRE)**: Percentage of total defects identified during testing vs. after deployment.
- **Defect Leakage**: Number of defects found in production compared to testing.
- **Test Case Effectiveness**: Ratio of test cases that found defects to total executed test cases.
- **Cost of Quality (CoQ)**: Total cost incurred in testing vs. the cost of post-release fixes.
- **Customer-reported Defects**: Number and severity of defects reported by users.
- **Post-mortem Analysis Metrics**: Insights gained from root cause analysis of major defects or failures.

# Defect Removal Efficiency (DRE)

- Percentage of defects that were found and removed before the software is released (or before moving to the next phase).
- It is one of the most important end-process metrics for evaluating the effectiveness of your quality assurance activities (reviews, inspections, testing).

$$\text{DRE} = \left( \frac{\text{Defects found before release}}{\text{Defects found before release} + \text{Defects found after release}} \right) \times 100\%$$

# Example

- Assume a project completed:
  - Defects found during development + testing = **120**
  - Defects reported by customers in first 6 months after release = **30**
- DRE $= \left(\frac{120}{120+30}\right) \times 100 = \frac{120}{150} \times 100 = 80\%$

80% of all defects were removed before release. 20% escaped to production.

# Defect Leakage

- Percentage of defects that escape from a current phase into a later phase (e.g., from development to testing, or from testing to production).

$$\text{Defect Leakage} = \left( \frac{\text{Defects found in later phase that originated in current phase}}{\text{Defects present in current phase (found in current + escaped to later)}} \right) \times 100\%$$

- **Why Use Defect Leakage Instead of DRE?**
  - **DRE** focuses on removal effectiveness (positive framing).
  - **Defect Leakage** focuses on what *escaped* (negative framing) – useful for root cause analysis and process improvement.

- Your project has two test phases: **System Testing** and **User Acceptance Testing (UAT)**.
- Defects found during System Testing = **80**
- After System Testing, UAT finds **20 defects** that should have been caught in System Testing (they are valid, not new requirements).
- Leakage from System Testing to UAT $= \left(\frac{20}{80+20}\right) \times 100 = \frac{20}{100} \times 100 = 20\%$
- **Interpretation:** 20% of the defects that existed in the system after System Testing escaped into UAT.

# Test Case Effectiveness

- Measures how well your test cases find defects.

$$\text{Test Case Effectiveness} = \left( \frac{\text{Number of test cases that failed (found} \geq 1 \text{ defect)}}{\text{Total number of test cases executed}} \right) \times 100\%$$

- **Failed test case** = A test case whose actual result differs from expected result, leading to one or more valid defect reports.
- **Executed test cases** = Test cases that were actually run (excluding blocked, skipped, or not yet executed).

- For accuracy
  - **Distinguish between test failure and defect** – A test case can fail because of a script error, bad test data, or environment glitch. Those should **not** count as "effective". Only failures that lead to a **valid, unique defect** count.
  - One defect per test case – If a test case finds three defects, it still counts as one effective test case (not three).
  - **Use with test case types** – Effectiveness varies by test type:
    - Smoke tests: usually 0–5% (they should pass)
    - Regression tests: 5–15% (if higher, product is unstable)
    - New feature tests: 20–40% (expect failures)
    - Negative/edge case tests: 30–60% (they are designed to fail)
  - **Normalize by test case priority** – A high-effectiveness low-priority test case may be less valuable than a medium-effectiveness critical-path test case.

# Cost of Quality (CoQ)

- This metric compares the cost incurred during testing with the cost of fixing defects after release

$$\text{CoQ Ratio} = \frac{\text{Cost of Testing}}{\text{Cost of Post-Release Fixes}}$$

- **Example:**
  - Cost of testing (test design, execution, internal bug fixes, test tools) = **$50,000**
  - Cost of post-release fixes (hotfixes, customer support, legal fees, lost sales) = **$200,000**
  - CoQ Ratio $= \frac{50{,}000}{200{,}000} = 0.25$------ For every $1 spent on testing, you avoid $4 of post-release fix costs.

# Customer-reported Defects

- Measures the number and severity of defects found by end users after release. Severity is typically classified into levels (e.g., Critical, High, Medium, Low).
- **Total customer-reported defects** = Count of unique defects reported by users in a period.
- **Defect rate per user** = (Total customer-reported defects) / (Number of active users) × 1000 (or per 1000 users).
- **Severity breakdown** = Number of defects per severity level.

# Example

- During one month after a mobile app release:
- Active users: 500,000
- Customer-reported defects: 47 total, broken down as:
  - **Critical** (app crash, data loss) → 3 defects
  - **High** (major feature broken) → 8 defects
  - **Medium** (minor UI glitch, workaround exists) → 22 defects
  - **Low** (cosmetic, typo) → 14 defects
- Defect rate per 1000 users = (47 / 500,000) × 1000 = **0.094 defects per 1000 users**
  Critical defect rate = (3 / 500,000) × 1000 = **0.006 critical defects per 1000 users**

# Post-mortem Analysis Metrics

- Used to capture insights from root cause analysis of major defects or failures.
- Post-mortem metrics turn a failure into measurable data that prevents recurrence. They focus on **root cause distribution**, **detection & resolution speed**, and **preventive action effectiveness**.

- **Root Cause Distribution (%)**
  - Root Cause % $= \frac{\textit{Number of incidents with a given root cause}}{\textit{Total major incidents}} \times 100$
- **Mean Time to Detect (MTTD) & Mean Time to Resolve (MTTR)**
  - MTTD $= \frac{\sum(\textit{Time from defect introduction to first detection})}{\textit{Number of major defects}}$
  - MTTR $= \frac{\sum(\textit{Time from detection to full resolution})}{\textit{Number of major defects}}$
- **Preventive Action Closure Rate (%)**
  - Action Completion Rate $= \frac{\textit{Number of completed preventive actions}}{\textit{Total identified preventive actions}} \times 100$

# Example

- Scenario: A cloud storage app experienced two major post-release failures in one quarter.
- **Failure A: File sync service outage (2 hours)**
  - Root cause (5-Whys): Missing timeout handling for third-party API → Code error
  - Time from code commit to detection: 5 days (detected by customer alert)
  - Time from detection to fix: 1.5 hours (hotfix + rollback)
- **Failure B: Data corruption on rename (affected 100 users)**
  - Root cause: Race condition in file rename transaction → Design flaw
  - Time to detection: 3 hours (internal monitoring)
  - Time to resolution: 8 hours (patch + data recovery)

- **List of preventive actions**
  - Add API timeout (completed)
  - Automate race condition test (completed)
  - Improve customer alerting (in progress)
  - Train team on concurrency (planned)

| Metric | Formula | Result |
| --- | --- | --- |
| **Root cause distribution** | Code error: 1 incident, Design flaw: 1 incident | Code 50%, Design 50% |
| **MTTD** | (5 days + 3 hours) ÷ 2 → convert to hours: (120h + 3h)/2 = 61.5 hours | **61.5 hours** (too long – improvement needed) |
| **MTTR** | (1.5h + 8h) ÷ 2 | **4.75 hours** |
| **Preventive actions identified** | 4 total:<br>- Add API timeout (completed)<br>- Automate race condition test (completed)<br>- Improve customer alerting (in progress)<br>- Train team on concurrency (planned) | Completion rate = 2/4 = **50%** |

# Activity

- **Unified Scenario: “FinApp” Mobile Banking Release**
- **Context:**
  - FinApp releases version 3.0. The test team executes a planned test suite. After release, customers use the app for one month. A post-mortem is conducted for major issues.
- **Given Data**
- **Testing phase (before release)**
  - Total test cases executed: **500**
  - Test cases that found at least one defect: **75**
  - Total unique defects found during testing: **60**

- **Post-release (customer usage – 1 month)**
  - Total unique defects reported by customers after release: 10
  - Severity breakdown of those 10:
  - Critical (app crash, data loss): 2
  - High (major feature broken): 3
  - Medium (minor UI glitch): 4
  - Low (typo): 1

- **Cost data (in birr)**
  - Cost of testing (test design, execution, tools, internal bug fixes): $90,000
  - Cost of post-release fixes (emergency patches, customer support, reputation loss): $360,000
- **Post-mortem analysis (for the 2 Critical post-release defects)**
  - Defect #1 (Critical): Transaction double-submit on slow network
    - Root cause: Missing idempotency key in API → Code error
    - Time to detect (TTD): 30 minutes (automated alert)
    - Time to resolve (TTR): 3 hours (hotfix + database cleanup)
  - Defect #2 (Critical): Face ID login failure for 5% of users
    - Root cause: Incomplete device compatibility test → **Test gap / process**
    - TTD: 2 days (customer support ticket)
    - TTR: 6 hours (patch + configuration update)

- Preventive actions identified from post-mortem:
  - Add idempotency check to all write APIs (completed)
  - Expand device lab with 20 more real devices (completed)
  - Implement canary deployment to catch device issues earlier (in progress)
  - Train testers on network fault injection (planned)
- → Total actions = 4; completed = 2